

CS599 大作业报告

多智能体交易竞技场：面向股票市场的语言驱动交易系统

课程名称：	CS599
项目名称：	多智能体交易竞技场：面向股票市场的语言驱动交易系统
方向：	Agentic AI 原生开发
学号：	请填写学号
姓名：	请填写姓名
专业：	请填写专业
指导老师：	请填写指导老师
提交日期：	2026 年 6 月 9 日

目录

一、选题背景与设计思想	3
二、Specs 规格文档	3
三、系统架构与设计	5
四、关键实现与代码展示	7
五、核心模块设计	10
六、测试与评估	11
七、系统升级与扩展	13
八、课程总结	14
九、运行说明	15
十、总结与展望	16

一、选题背景与设计思想

问题定义

本项目关注的问题是：如何构建一个可运行、可观察、可复盘的语言驱动多智能体股票交易仿真系统。具体而言，系统需要让多个投资者 Agent 在同一虚拟股票市场中根据资金、持仓、市场信息、传闻和自身投资原则产生买卖决策，并通过市场撮合机制真实改变订单状态、股票价格和个人财富。

现有方案不足或原始系统分析

传统交易仿真常使用固定规则、随机策略或统计模型描述投资者行为，优点是可控和易复现，但难以表达自然语言信息、传闻传播、投资者个性和策略反思。纯 LLM 文本推演虽然能生成丰富叙述，但通常缺少账户、持仓、订单簿和价格约束，容易产生不可执行交易。本项目原始系统已具备 Django、Vue 和 Stockagent 仿真雏形，但文档、敏感配置、安全边界和规格描述仍需补齐。

项目价值

项目价值体现在三个方面：第一，将 LLM 生成能力嵌入有状态的交易市场，使 Agent 决策受到现金、持仓和撮合规则约束；第二，通过 SQLite 持久化记录订单、股票、账户、记忆和传闻，便于评阅、展示和复盘；第三，通过前端页面展示参数配置、财富曲线、股票行情和 Agent 对比，使 Agentic AI 的运行过程具备可解释性和可观察性。

技术路线

技术路线采用前后端分离加仿真核心的结构。前端使用 Vue 3、Vite、Element Plus 和 ECharts 完成参数输入和结果展示；后端使用 Django 提供 API；Stockagent 模块负责 Agent、股票、市场指数、Broker、订单撮合和数据库写入；LLM 调用通过 OpenAI SDK 兼容接口封装，并通过环境变量配置 API Key；仿真结果写入 SQLite，再由 API 转换为 JSON 返回前端。

二、Specs 规格文档

Product Spec

产品目标是提供一个多智能体股票交易竞技场。目标用户包括课程评阅者、项目开发者和对 Agentic AI 金融仿真感兴趣的实验人员。核心用户流程为：打开前端首页，设置模拟天数、Agent 数量和股票数量，启动仿真，等待后端完成 LLM 决策与市场撮合，然后查看 Agent 交易详情、股票价格走势和 Agent 收益对比。产品成功标准包括：参数可配置、仿真可运行、结果可展示、数据可复盘、敏感信息不硬编码。

Architecture Spec

系统采用四层架构：展示层、API 层、仿真领域层和数据/外部服务层。展示层由 Vue 页面和组件组成；API 层由 Django urls.py 与 views.py 组成；仿真领域层由 main.py、Person.py、Stock.py、Market.py、behavior.py 和 database_utils.py 组成；数据层使用 SQLite 保存仿真结果，Redis 作为缓存或外部状态测试工具，LLM 服务作为外部推理提供方。各层之间通过 HTTP API、函数调用和数据库读写解耦。

API Spec

后端主要 API 包括 GET /api/paraminit、GET /api/simulation_results、GET /api/stock_data、GET /api/agent_comparison 和 GET /redis_test。/api/paraminit 接收 No_Days、Num_Person、Num_Stock，返回仿真状态、保存路径和 simulation_id；/api/simulation_results 返回 agents、profitCurve 和 actions；/api/stock_data 返回股票 OHLC 价格序列；/api/agent_comparison 返回 Agent 收益曲线、最终资产和日期标签。所有业务接口以 JSON 形式返回 status、message 和 data 字段。

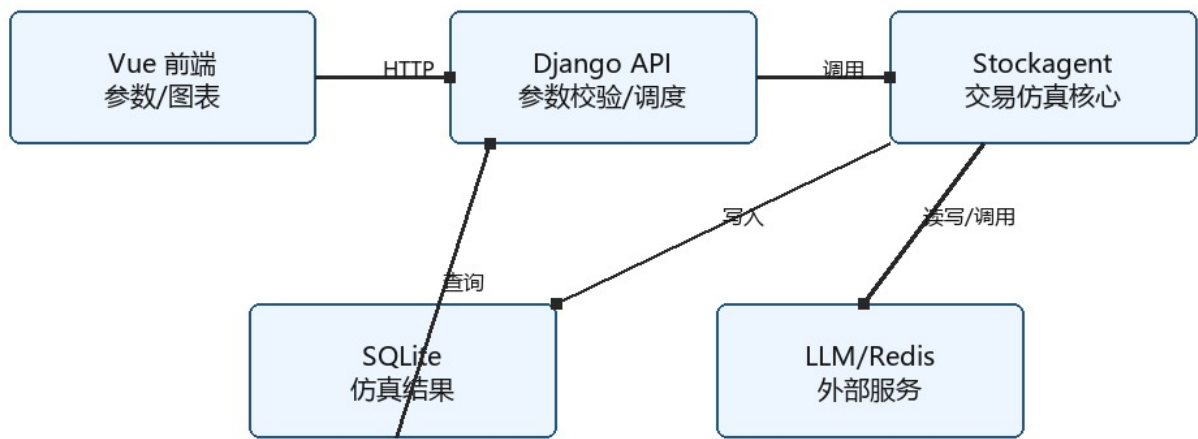
数据规格

核心数据表包括 active_orders、stock、person、account、memory 和 gossip。active_orders 保存订单时间戳、股票编号、人员编号、买卖类型、价格、数量和状态；stock 保存股票每日开盘、收盘、最高、最低、成交量等数据；person 保存现金、资产、财富、收益和投资原则；memory 保存 Agent 的交

易操作、策略、分析和财务上下文；gossip 保存市场传闻。

三、系统架构与设计

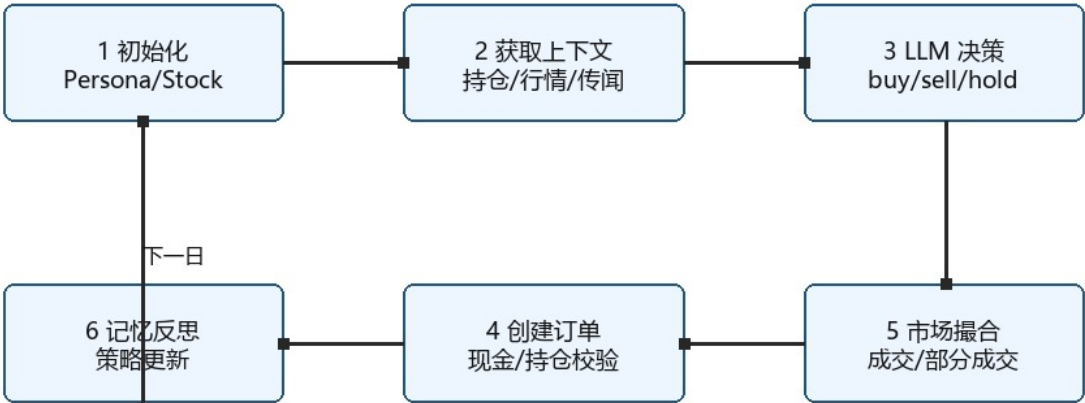
核心架构图



说明：前端展示、后端接口、仿真核心、数据与外部服务四层划分。

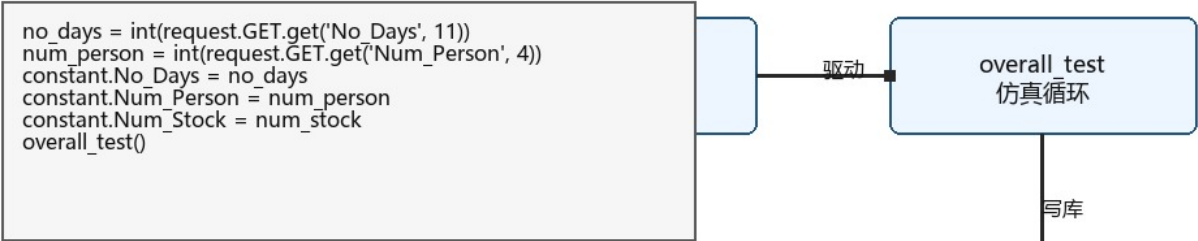
每层职责单一，便于展示、调试和复盘。

Agent 交互流程

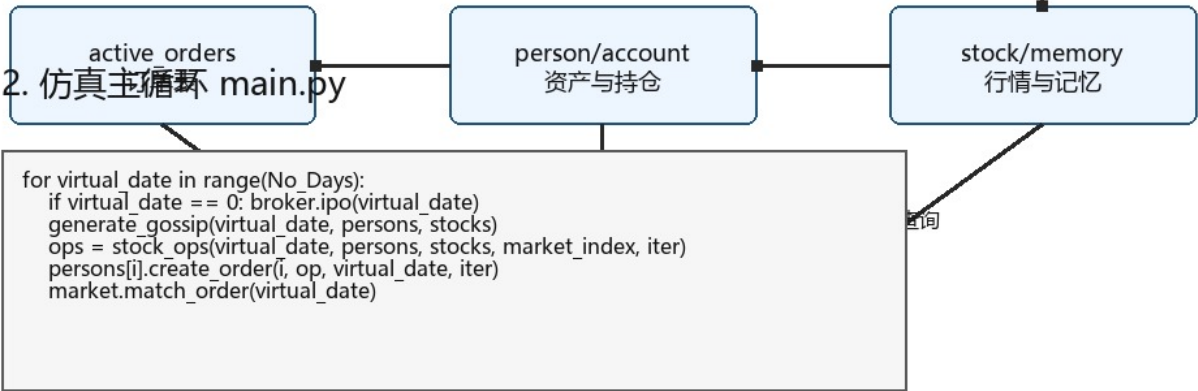


说明：Agent 文本决策需经过订单解析、账户约束和市场撮合。
只有成交结果会改变价格与财富，避免纯文本推演脱离状态。

数据流仿真入口 views.py



说明：API 接收前端参数，更新仿真常量并启动主流程。



说明：每日完成信息生成、Agent 行为、订单创建和市场撮合。

说明：数据从参数进入仿真，从仿真写入 SQLite，
再由结果 API 聚合为前端图表数据。

四、关键实现与代码展示

3. Agent 下单约束 Person.py

```
if order_type == 'buy':
    cash_expense = bid_price * quantity
    if cash_expense < self.cash:
        submit_order(self.db, order_type, self.person_id, stock_id, ...)
if order_type == 'sell':
    stock_onhold = self.query_single_stock(virtual_date, stock_id)
    if stock_onhold and stock_onhold['quantity'] >= quantity:
        submit_order(self.db, order_type, self.person_id, stock_id, ...)
```

说明：LLM 输出不会直接成交，必须先通过现金和持仓校验。

4. 市场撮合 Market.py

```
buy_orders = self.fetch_orders('buy', stock_iter)
sell_orders = self.fetch_orders('sell', stock_iter)
deal_price = (current_buy['price'] + current_sell['price']) / 2
trade_quantity = min(current_buy['quantity'], current_sell['quantity'])
self.stocks[stock_iter].update_trade_data(today, cur_stock_price, trade_quantity)
self.update_order(current_buy, deal_price, 'finished', trade_quantity)
```

说明：撮合模块决定成交价、成交量、订单状态和价格更新。

5. 安全配置环境变量

```
openai.api_key = os.getenv('OPENAI_API_KEY', '')
client = openai_client('DASHSCOPE_API_KEY', 'DASHSCOPE_BASE_URL', ...)
host = os.getenv('REDIS_HOST', 'localhost')
password = os.getenv('REDIS_PASSWORD')
SECRET_KEY = os.getenv('DJANGO_SECRET_KEY', 'dev-only-insecure-secret-key-change-me')
```

说明：敏感凭据统一由环境变量注入，避免硬编码泄露。

6. Agent 核心循环

```
for virtual_date in range(No_Days):
    market_index.update_market_index(virtual_date)
    generate_gossip(virtual_date, persons, stocks)
    ops = stock_ops(virtual_date, persons, stocks, market_index, iter)
    for i in rand:
        persons[i].create_order(i, op, virtual_date, iter)
    market.match_order(virtual_date)
    each_person.end_of_iteration(virtual_date, iter)
```

说明：核心循环体现 “感知-决策-行动-撮合-记忆更新” 。

7. 工具定义：LLM 与 Redis

```
def _openai_client(api_key_env, base_url_env=None, default_base_url=None):
    kwargs = {'api_key': _require_env(api_key_env)}
    if base_url: kwargs['base_url'] = base_url
    return OpenAI(**kwargs)

class RedisService:
    def set_value(self, key, value): return self.r.set(key, value)
    def get_value(self, key): return self.r.get(key)
```

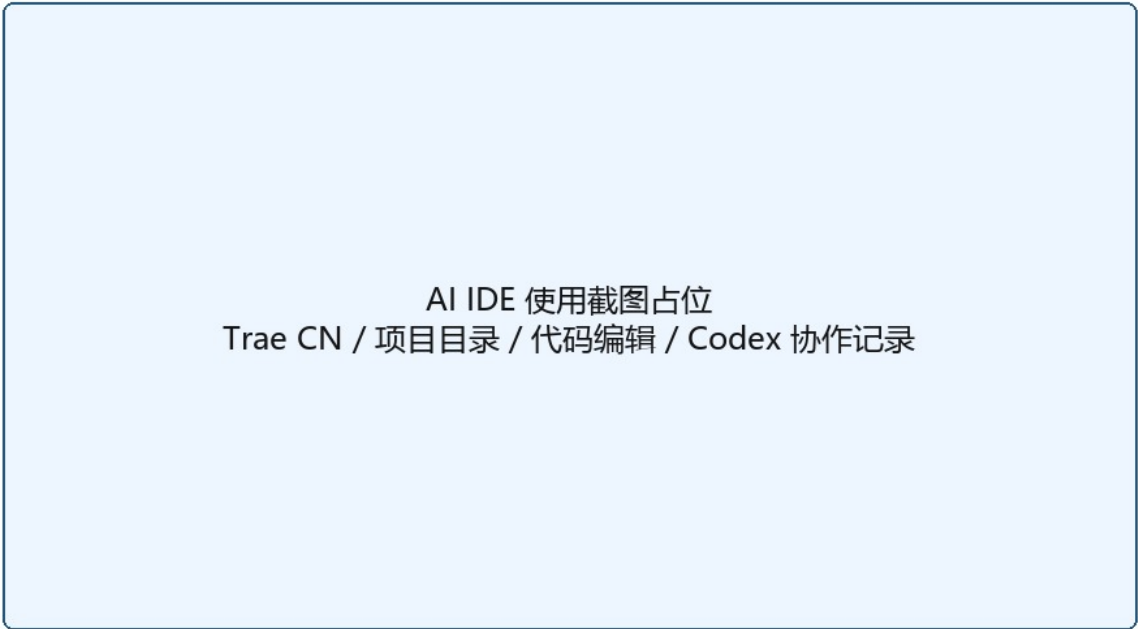
说明：工具层封装模型服务和缓存服务，Agent 行为通过工具接入外部能力。

8. 配置文件

```
No_Days = 3
Num_Person = 4
Num_Stock = 3
SAVE_NAME = 'sim01'
DASHSCOPE_API_KEY=
REDIS_HOST=localhost
DJANGO_SECRET_KEY=
```

说明：运行参数在 constant.py 中维护，敏感参数在 .env.example 中声明。

9. AI IDE 使用截图



说明：提交前可将该占位替换为 Trae CN 中打开 AgentStock 项目的实际截图。

五、核心模块设计

Agent 与 Broker

Person 类表示投资者 Agent，维护现金、持仓、资产、财富、投资原则、最低生活成本和反思频率。Broker 作为特殊参与者负责 IPO 和市场流动性补充，并在交易完成后进行资产结算。

股票与市场指数

Stock 类维护股票名称、当前价格、发行数量、每股分红、日内价格序列和历史价格数据。Market_index 根据股票账面价值权重计算市场指数，用于向 Agent 提供市场整体变化信息。

市场撮合机制

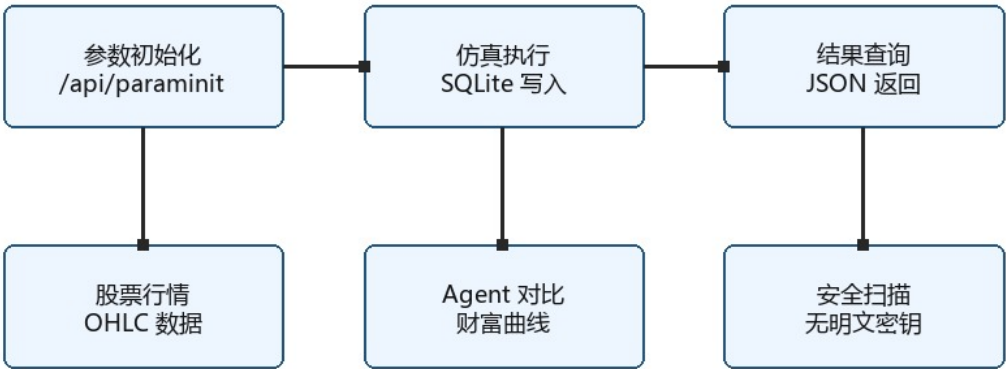
Market 类负责订单撮合，从 active_orders 表中读取买卖订单，对同一股票的买卖双方进行匹配。成交后系统更新订单状态、成交数量、个人账户、Broker 库存和股票价格。

LLM 行为生成

behavior.py 将 LLM 输出转换为可执行行为。系统先生成股票分析和传闻，再生成买入、卖出或持有建议，并用正则解析为可执行订单。

1. 功能测试

六、测试与评估



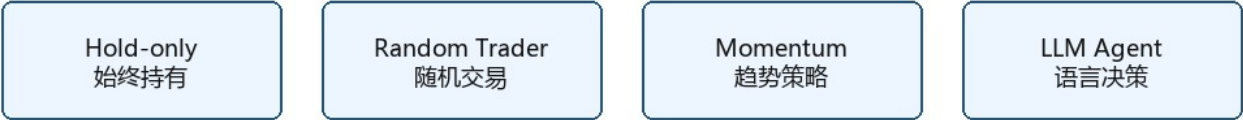
说明：功能测试覆盖参数、仿真、查询、展示和安全配置。

2. Agent 行为评估



说明：Agent 输出需可执行、受账户约束，并能在 memory 中复盘。

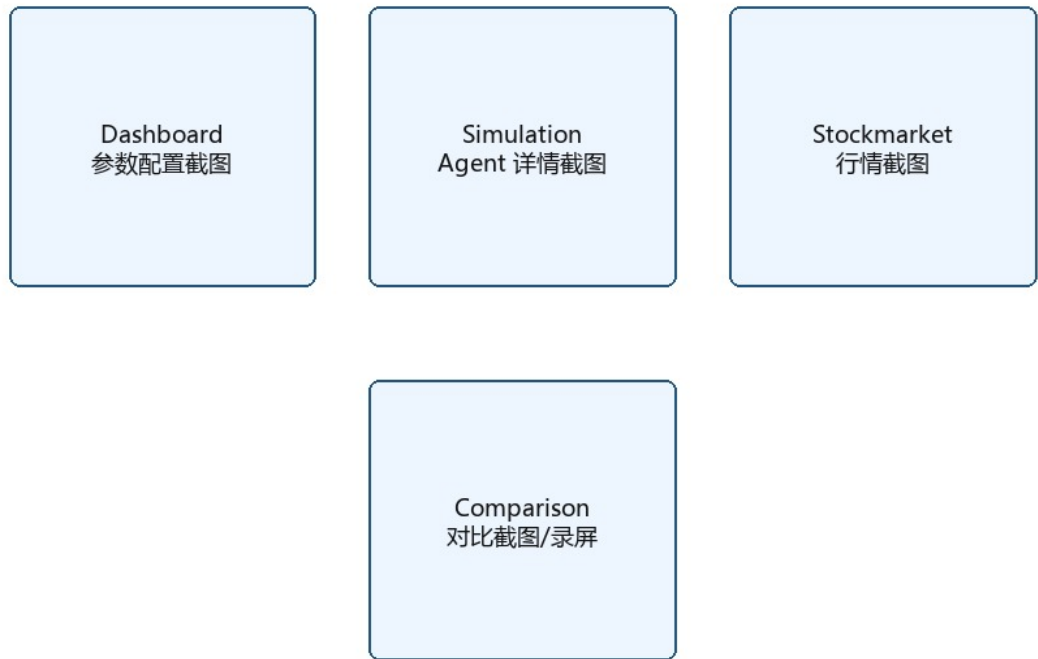
3. Benchmark



指标：最终财富 / 累计收益率 / 最大回撤 / 交易次数 / 有效订单率 / 响应耗时

说明：当前系统具备记录订单和财富曲线的基础，可扩展为多轮统计 benchmark。

4. Demo 截图/录屏



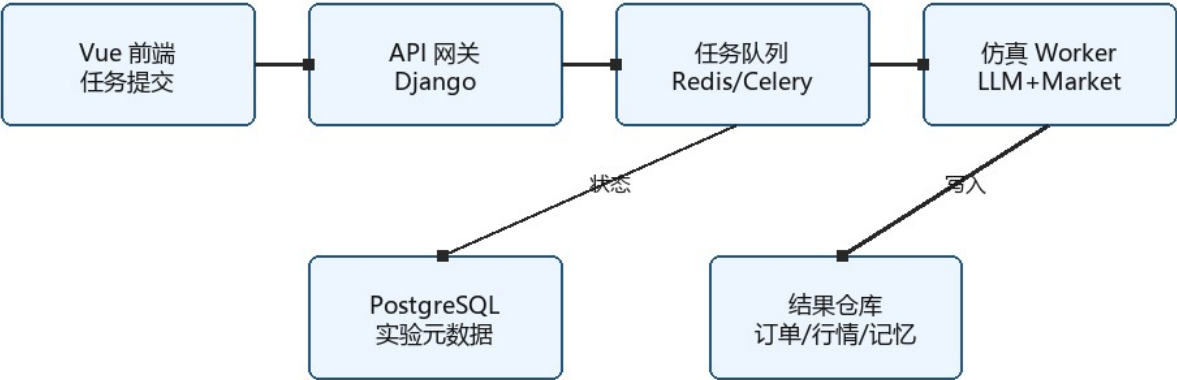
说明：提交前可替换为浏览器或 Trae CN 中实际 Demo 截图；录屏文件可命名为 demo.mp4。

5. 不足与改进

后续补充 pytest、Playwright、SQLite 数据断言和多轮 benchmark 统计。

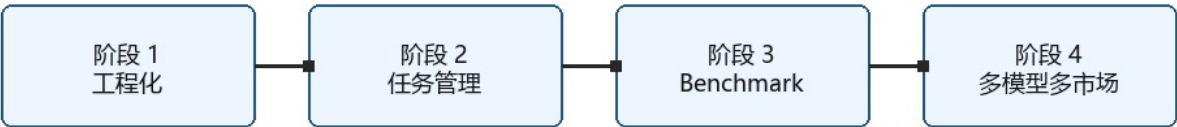
七、系统升级与扩展

1. 可扩展架构



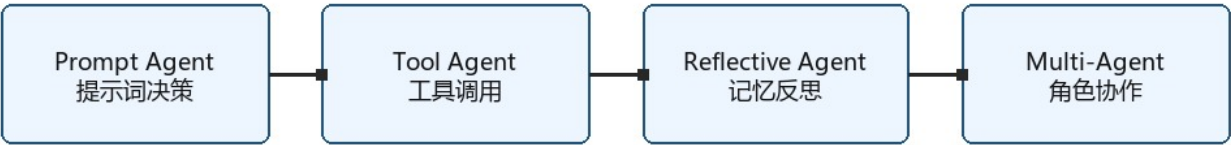
说明：同步仿真升级为异步任务，可支持长任务、并发仿真和历史实验管理。

2. 下一阶段计划



说明：先补工程质量，再做任务管理，最后扩展评测和多模型能力。

3. AI 能力演进路径



Prompt: 生成 buy/sell/hold; Tool: 接入行情/持仓/风险工具;
Reflective: 长期记忆与策略复盘; Multi-Agent: 研究员/交易员/风控员协作。

4. 扩展风险与控制



说明：扩展同时引入工程风险，需要用日志、版本、校验和限流控制。

八、课程总结

个人收获

通过本课程大作业，我从单纯调用大模型生成文本，转向理解如何把 LLM 放入一个有状态、有约束、有反馈的真实软件系统中。我不仅学习了 Django、Vue、SQLite 和前后端接口协作，也更清楚地理解了 Agent 在 AI 中目标、工具、状态、记忆和评估之间的关系。

工程思维转变

过去更关注功能是否能跑通，现在更关注系统是否可维护、可复现、可评估和可扩展。本项目让我意识到，真正的工程交付不仅包括代码，还包括规格文档、架构设计、安全配置、测试评估、运行说明和后续演进路线。AI IDE 可以提高效率，但不能替代开发者对边界、风险和质量的判断。

对课程的建议

建议课程后续增加三个实践环节：第一，提供一份标准大作业报告模板，让学生更早按评分项组织项目；第二，增加 AI IDE 与 Agent 工程化的专项示例，例如如何做工具调用、记忆管理和 benchmark；第三，安排一次中期代码评审或 Demo Review，帮助学生提前发现依赖、部署、测试和安全问题。

后续自我提升

后续我希望继续提升三方面能力：系统化设计能力，先写清 Product Spec、Architecture Spec 和 API Spec 再实现；测试与评估能力，为 Agent 行为建立 benchmark 和自动化验证；AI 工程能力，把 LLM 从提示词调用升级为具备工具、记忆、反思和多角色协作的 Agentic workflow。

九、运行说明

运行环境

后端运行目录为 src/AgentStock，启动命令为 python manage.py runserver。前端运行目录为 src/AgentStock/Fronted，启动命令为 npm run dev。前端默认访问 localhost:5173，后端默认监听 localhost:8000。

配置要求

完整 LLM 仿真需要配置 DASHSCOPE_API_KEY、DEEPINFRA_API_KEY、GOOGLE_API_KEY 或 OPENAI_API_KEY 等环境变量；Redis 测试接口需要 REDIS_HOST、REDIS_PORT、REDIS_PASSWORD 和 REDIS_DB。

十、总结与展望

总结

本项目实现了一个以语言模型驱动的多智能体股票交易仿真平台。系统将 Agent 决策、市场撮合、数据记录和 Web 可视化连接起来，使交易行为能够由自然语言信息、历史记忆和策略反思共同影响。

展望

后续可以增强实验可复现性，为每次仿真保存完整参数、模型配置和随机种子；改进前端结果管理，支持多次仿真横向比较；引入最大回撤、夏普比率、换手率等风险指标，使系统更接近真实量化研究流程。